

Building GPT-2 from Scratch: What We Learned

Hasan Semih Selçuk Matin Huseynzade

Dept. of Computer Engineering, IZTECH. CENG-534 Default Final Project

We set out to do two things: understand GPT-2 by rebuilding it from its primitives, and then learn what it takes to bend a single pretrained model toward tasks of increasing difficulty. In hindsight the project taught us less about any one accuracy number and more about a recurring distinction between *recognizing* a property of text and *producing* text that has it. It also taught us how much of practical NLP lives in details that have nothing to do with the model itself. This note records what we take away rather than a slide-by-slide account; the full report carries every table and command.

Task	Setup	Result (dev / held-out)
Sentiment (CFIMDB)	full-model fine-tune	accuracy 0.967 (vs. 0.857 head-only)
Sentiment (SST-5)	full-model fine-tune	accuracy 0.513
Paraphrase (Quora)	cloze next-token, 3 epochs	accuracy 0.889
Sonnet generation	temperature sweep, top- p 0.9	chrF 41.49 at $\tau = 1.0$
Extension (mod. $\rightarrow$ Shak.)	fine-tune + source-side masking	chrF 35.83 (13.85 zero-shot, +21.97)

Table 1: Our results on the course dev / held-out splits. Every number is our own measurement and reproduces from committed artifacts (see end).

Building the model was the cheap part; adapting it was the lesson. Writing masked attention, the pre-LayerNorm block, and a decoupled-weight-decay AdamW by hand, and then watching the course tests pass, was satisfying but mechanical. The interesting work began at fine-tuning. Our four tasks form a deliberate progression (sentiment classification, paraphrase detection, sonnet generation, and our extension), and each step up that ladder forced a new kind of decision. Classification only asks the model to read. Generation asks it to commit to an output distribution token by token, where small choices compound. The same backbone served all four, which we read as evidence that the value of a pretrained model is less in any task head and more in what transfers underneath.

Paraphrase: the framing mattered more than the tuning. We treated the binary Quora task not as a classifier with a new head but as a fill-in-the-blank question, where the model simply prefers *yes* or *no* as the next token, and reached 0.889 dev accuracy. What stayed with us was not the number but two quiet failure modes. Using even a slightly different prompt at test time than at training time made dev look healthy while the held-out score quietly collapsed. And naively truncating long inputs from the right threw away the very end of the prompt, where the answer is decided. Both bugs are invisible in the loss curve. The lesson we drew is that when a task is posed as language modeling, the prompt *is* part of the model, and inconsistencies in it are as damaging as a wrong hyperparameter.

Generation taught us to read a metric skeptically. For sonnets we condition on three lines and sample the rest, and a temperature sweep reproduced, on our own model, the textbook degeneration/divergence trade-off: too cold and the model loops, too hot and it wanders, with $\tau=1.0$ best at chrF 41.49. But the gaps between settings were small enough that we stopped treating chrF as ground truth. chrF rewards n -gram overlap, exactly the signal that over-confident, repetitive text can inflate, so we cross-checked with held-out perplexity, which fine-tuning roughly halved (99.1 $\rightarrow$ 59.0), and with corpus-level diversity, which moved

in the direction the chrF trend implied. The habit we are taking forward is to never let a single generation metric stand alone.

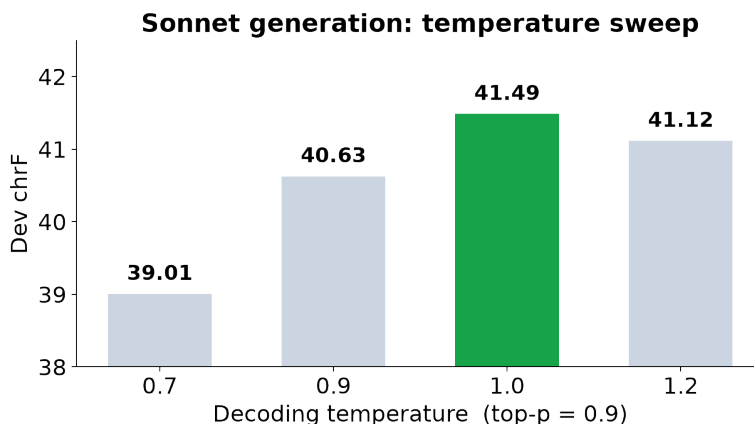


Figure 1: Sonnet decoding-temperature sweep (dev chrF, top- p fixed at 0.9). The ordering is clear but the spread is narrow, which is why we did not over-read it.

The extension: a small idea, isolated cleanly. Our original contribution rewrites modern English into Shakespearean style. We deliberately introduced *no new model class*: we reused the sonnet generator and changed only the data and the loss, masking the source so the gradient comes from the target tokens alone. This was a methodological choice as much as an engineering one. By holding the architecture fixed, any improvement is attributable to data and objective, not to a lucky new module. It worked: chrF more than doubled over the zero-shot baseline ($13.85 \rightarrow 35.83$), and the outputs adopt archaic morphology while preserving meaning, with failures dominated by harmless sampling repetition rather than semantic drift.

The most useful finding came from the tokenizer, not the network. The single change with the highest return per character was deleting one space. GPT-2’s byte-pair encoding binds a leading space to the following token, so a prompt ending in a space pushes the very first generation step out-of-distribution; our model responded with strings of junk before recovering. Moving that space from the prompt onto the target was worth +6.3 chrF. We highlight this because it reframes where effort pays off. Understanding the tokenizer’s conventions returned more than any tuning we did to the model itself, and it is the kind of bug that no validation accuracy would have flagged.

Hardware was a real constraint, and it disciplined our claims. Everything was trained on an Apple M4 Pro with no CUDA GPU, on the MPS backend, where variable-length batches fragment the allocator and eventually crash. The recipe that made full fine-tuning stable (bounding sequence length, small batches, periodic cache clearing, evaluation under `no_grad`) is not novel, but living under that ceiling changed how we reported results. The clearest example is the one place we trail the handout (CFIMDB full, 0.967 vs. 0.976). The tempting story is that our memory-forced small batch caused it. We tested that story with a controlled gradient-accumulation experiment and it did not hold: a larger effective batch did not close the gap within the same budget. So we report the gap as unexplained rather than blame a convenient culprit. Testing a hypothesis and finding it false felt like the most honest paragraph in the work.

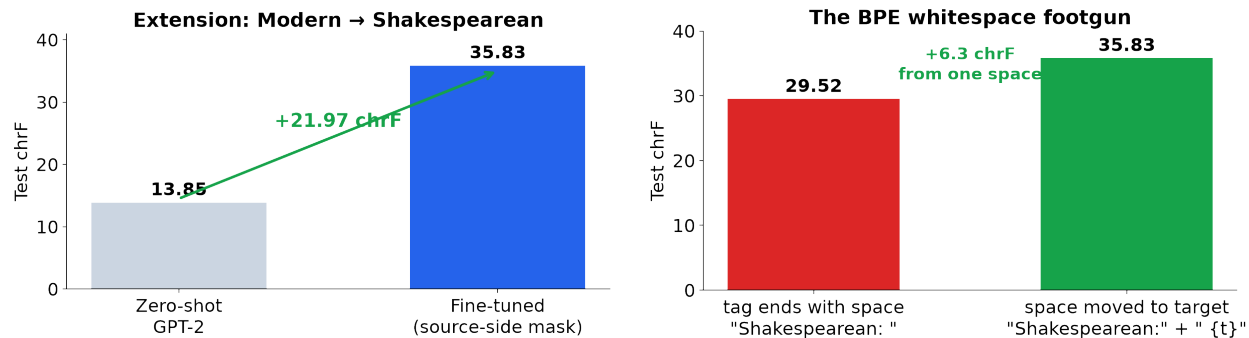


Figure 2: Left: the extension more than doubles chrF over zero-shot GPT-2. Right: of the fine-tuned gain, 6.3 chrF came from a single space moved off the prompt tag, the tokenizer finding above.

What we would not claim, and what we would do next. All our numbers come from single runs, so we treat small differences as noise and would re-run with multiple seeds and report variance before asserting any fine ranking. The sonnet corpus is tiny and overfits within a couple of epochs. The natural next step, and the one our hardware kept pointing at, is parameter-efficient fine-tuning such as LoRA, which would relieve both the memory pressure and the overfitting and let us move to a larger backbone. Beyond that, preference-based tuning for sonnet quality and a richer generation metric such as MAUVE would sharpen the evaluation. If the project has a single takeaway, it is that the distance between a model that works and one that works *well* was paved, for us, with prompts, tokenizers, and memory, not with the architecture we started by building.

Reproducibility. Every number is regenerable in seconds from committed artifacts, with no retraining, via `reproduce_metrics.py`, which re-scores the saved predictions against the gold labels.

Code, full report, and reproduction scripts: https://github.com/chillmatin/cs224n_gpt